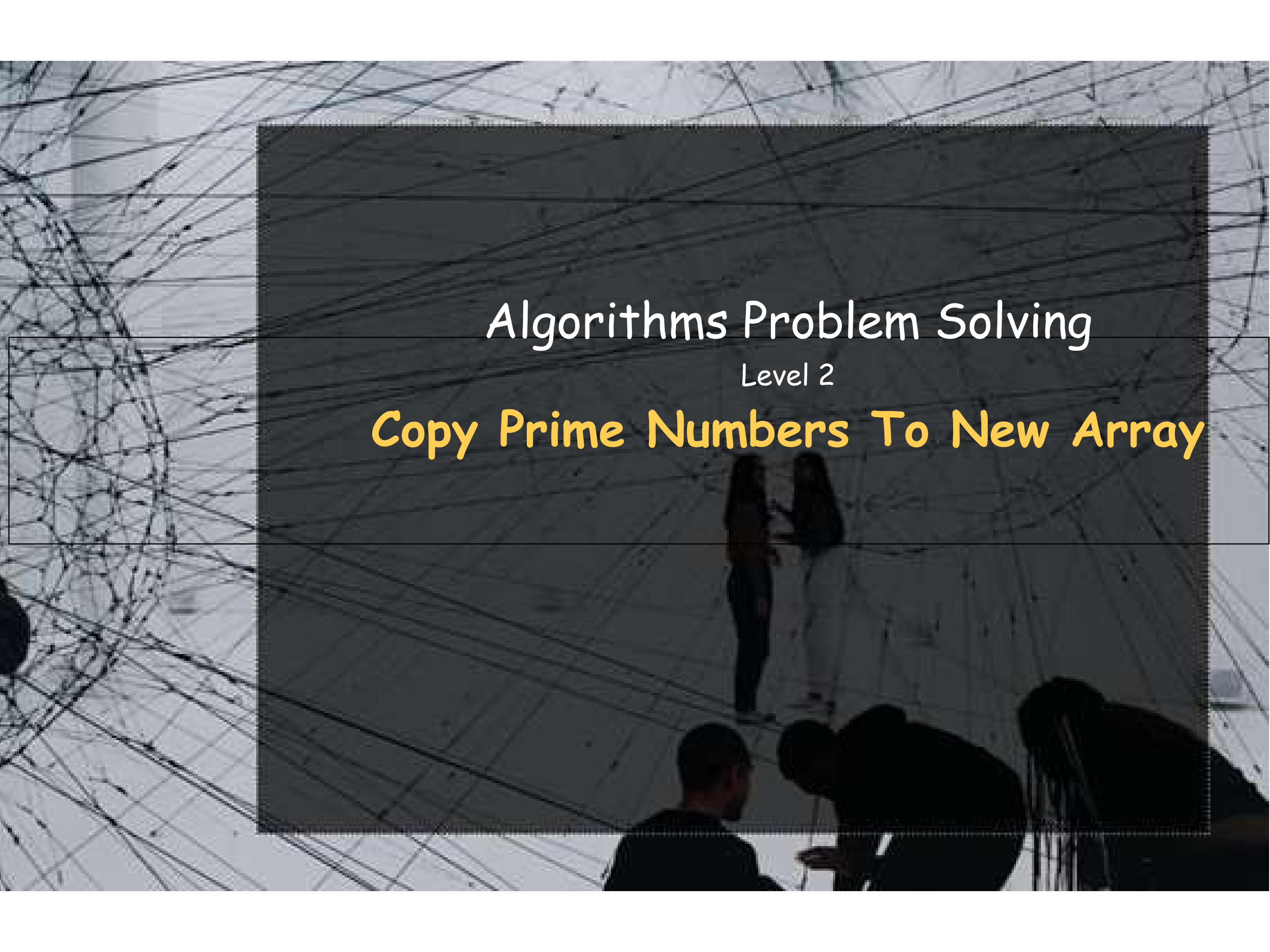


Algorithms Problem Solving

Level 2

Copy Prime Numbers To
New Array



Algorithms Problem Solving

Level 2

Copy Prime Numbers To New Array

Definition

- ◆ This program fills an array with random numbers from `1` to `100`.
- ◆ It checks each number to determine whether it is prime.
- ◆ Only the prime numbers are copied into a second array.
- ◆ The copying is done using `AddElementToArray()`.
- ◆ Finally, the original array and the array containing the prime numbers are printed.

Explanation

- ◆ `CheckPrime()` checks whether a number is prime.
- ◆ It starts checking possible divisors from `2` up to half of the number:

```
for (int counter = 2; counter <= M; counter++)
```
- ◆ If the number can be divided evenly by any of these values, it is not prime:

```
if (Number % counter == 0)
{
    return enPrimeNotPrime::NotPrime;
}
```
- ◆ If no divisor is found, the function returns `Prime`.
- ◆ `CopyPrimeNumbers()` loops through all elements of `arr1`.
- ◆ If an element is prime, it is added to `arr2` using:

```
AddElementToArray(arr1[i], arr2, arr2Length);
```
- ◆ `AddElementToArray()` increases `arr2Length` and stores the prime number at the next available position.

Steps in main()

1. Create two arrays with a maximum size of `100`.
2. Set `arr1Length` and `arr2Length` to `0`.
3. Fill `arr1` with random numbers from `1` to `100`.
4. Print `arr1`.
5. Loop through all elements of `arr1`.
6. Check each number using `CheckPrime()`.
7. If the number is prime, call `AddElementToArray()`.
8. Add the prime number to `arr2`.
9. Increase `arr2Length`.
10. Print `arr2`.

💡 Example

◆ `Array 1` :

42 68 35 1 70 25 79 59 63 65

◆ The program checks each number:

42 → Not Prime

68 → Not Prime

35 → Not Prime

1 → Not Prime

70 → Not Prime

25 → Not Prime

79 → Prime

59 → Prime

63 → Not Prime

65 → Not Prime

◆ Therefore, the prime numbers are:

79 59

➡ Result

Array 1 Elements: 42 68 35 1 70 25 79 59 63 65

Array 2 Prime Numbers: 79 59

✚ Note

◆ The enum:

```
enum enPrimeNotPrime { Prime = 1, NotPrime = 2 };
```

◆ makes the result of `CheckPrime()` easier to understand.

◆ `CopyPrimeNumbers()` is responsible for filtering the array.

◆ `AddElementToArray()` is responsible for adding the selected numbers to the second array.

◆ `arr2Length` starts at `0` and increases only when a prime number is found.

◆ The function currently uses:

```
int M = round(Number / 2);
```

◆ Since `Number` and `2` are integers, `Number / 2` is integer division before `round()` is applied. For this program, that still gives the needed half value.

⊘ Warning

- ◆ There is an important bug in `CheckPrime()`: the number `1` is incorrectly considered prime because the loop does not run for `1`.
- ◆ A prime number must be greater than `1`.
- ◆ The function should start with:

```
if (Number <= 1)
    return enPrimeNotPrime::NotPrime;
```
- ◆ A more efficient prime check can stop at `Number * Number <= counter` instead of checking up to half the number.
- ◆ The program should ensure that the entered array length does not exceed `100`.
- ◆ `ReadArrayElements()` is not used.
- ◆ `cmath` and `corecrt_math_defines.h` are unnecessary for the rest of the program.
- ◆ `srand()` and `time()` should ideally have their proper headers included explicitly.



Comparison

▣ `CheckPrime()`

- ✓ Determines whether one number is prime.
- ✓ Returns `Prime` or `NotPrime`.
- ✓ Does not modify the arrays.

▤ `CopyPrimeNumbers()`

- ✓ Checks all elements in the first array.
- ✓ Uses `CheckPrime()` to identify prime numbers.
- ✓ Copies only prime numbers to the second array.
- ✓ Uses `AddElementToArray()` to add each prime number.

✦✦ Summary

- ✓ Fill the first array with random numbers.
- ✓ Check every number to determine whether it is prime.
- ✓ Select only prime numbers.
- ✓ Use `AddElementToArray()` to add them to the second array.
- ✓ `arr2Length` keeps track of the number of copied prime elements.
- ✓ Remember that `1` is not a prime number.

Thank you 😊